

Judicial Case Management System (JCMS)

Complete Blueprint: Core Architecture + AI/LLM Integration

Contents

- 1. Feature Scope & Functional Requirements
- 2. Architecture & Tech Stack Recommendations
- 3. Backend API Endpoint Design
- 4. Standout Portfolio Enhancements
- 5. Phase-by-Phase Development Plan
- 6. AI/LLM Integration — Recommended Features & Use Cases
- 7. AI/LLM Integration — Technical Architecture (Models, Vector DB, Guardrails)

1. Feature Scope & Functional Requirements

1.1 Judge Dashboard

Feature	Description
Hearing Docket View	Daily/weekly cause list auto-sorted by priority score; drag-to-reorder within permitted bounds
Case Disposition	Record orders, adjournments, next hearing dates, final judgments; attach signed order PDF
Courtroom Management	View/allocate courtroom + time slot; flag conflicts (double-booking, judge unavailability)
Case File Viewer	Consolidated view of petition, evidence, prior orders, and full timeline for a case
Emergency Queue Alerts	Real-time badge/notification when an urgent petition is filed and auto-routed to their bench
Bench Notes	Private judge-only notes per case (not visible to advocates/registrar)

1.2 Advocate/Lawyer Portal

Feature	Description
e-Filing	Submit new petitions/case filings with structured metadata (case type, parties, relief sought)
Evidence & Document Upload	Multi-file upload (PDF/images), versioned, virus-scanned, checksum-verified
Emergency Petition Flag	Mark a filing as urgent with justification; triggers expedited registrar review
Case Tracking	Status timeline: Filed → Under Verification → Listed → Heard → Disposed
Hearing Calendar	Personal calendar of all cases across courts, with conflict warnings for double-booked slots
Notifications	Email/SMS/push for hearing date changes, registrar objections, orders passed

1.3 Registrar/Court Staff Portal

Feature	Description
Filing Verification	Checklist-driven scrutiny (fee paid, format compliant, jurisdiction correct); accept/return-for-correction
Scheduling Engine Console	Run/override the auto-scheduler; manually resolve conflicts it flags
Queue Management	Visual priority queue per court/judge; manual re-prioritization with mandatory justification (logged)
Emergency Override Workflow	Fast-track intake → immediate registrar triage → same-day/next-slot bench assignment
Cause List Publishing	Generate and publish the next day's cause list (PDF + web)
Staff Task Queue	Assigned verification/administrative tasks with SLA timers

1.4 Public/Litigant Portal

Feature	Description
Public Cause List	Searchable, filterable, no-login list of hearings by court/date
Case Status Lookup	Search by case number/CNR-equivalent to see current stage (no sensitive documents exposed)
Order/Judgment Access	Download publicly available final orders (redacted per privacy rules)
Court/Judge Directory	Static reference info (courtroom numbers, sitting hours)

1.5 Cross-Cutting Standout Features

- Weighted Priority Scheduling Algorithm** — composite score from: case age (aging bonus), statutory urgency class (e.g., bail, habeas corpus > civil), emergency flag, court-imposed deadlines, and litigant vulnerability flags (e.g., senior citizen, POCSO). Recomputed on every schedule run; fully explainable (score breakdown visible to registrar/judge).
- Emergency Case Override Workflow** — a state machine: Filed(Emergency) → Registrar Triage (SLA: 2h) → Judge Acceptance → Slot Injection (bumps/reflows queue) → Notification fan-out. Every override is logged with actor, reason, and before/after queue snapshot.
- Digital Audit History** — immutable, append-only audit log (hash-chained) for every state-changing action: filings, verifications, reassignments, order uploads, overrides. Exportable per-case as a tamper-evident PDF trail.

2. Architecture & Tech Stack Recommendations

2.1 Recommended Stack

Layer	Choice	Why
Frontend	React (Vite) + TypeScript + TailwindCSS + shadcn/ui	Fast dev loop, strong typing, component reuse across 4 portals via shared design system
State/data	TanStack Query + Zustand	Server cache + minimal client state, avoids Redux boilerplate
Backend	Node.js + NestJS (TypeScript)	Opinionated modular architecture (great for role-based portals + DI + guards for RBAC), or Spring Boot (Java) if you want to signal enterprise/backend depth for recruiters targeting Java shops
Database	PostgreSQL	Strong relational integrity for case/hearing relationships; supports JSONB for flexible metadata, row-level security for RBAC enforcement at DB layer
ORM	Prisma (Node) or JPA/Hibernate (Java)	Type-safe schema, migrations
Auth	JWT (access + refresh) via Keycloak or Auth0 , or self-rolled with Passport.js + bcrypt/argon2	Keycloak looks great on a resume (OIDC, RBAC federation) and demonstrates real IAM knowledge
Caching	Redis	Cache hot cause-lists, session store, rate limiting, pub/sub for live updates
Queue	Redis Streams / BullMQ , or RabbitMQ for a more "enterprise" signal	Async jobs: document virus scan, PDF generation, notification fan-out, priority recompute
Real-time	WebSockets (Socket.IO) or Server-Sent Events	Live docket updates, emergency alerts, queue reordering pushed to judges/registrars instantly
File storage	S3-compatible (AWS S3 / MinIO for local dev)	Evidence/document storage with signed URLs, not DB blobs
Search	PostgreSQL full-text search (MVP) → Elasticsearch/OpenSearch (Phase 3)	Public case-status lookup and advocate search at scale

Layer	Choice	Why
Infra	Docker Compose (dev) → Kubernetes/ECS (prod), Terraform for IaC	Cloud-ready signal for recruiters
CI/CD	GitHub Actions	Lint/test/build/deploy pipeline, container publish
Observability	Prometheus + Grafana, Sentry for error tracking	Production-readiness signal

2.2 Relational Database Schema (Core Entities)

```
User (id, name, email, password_hash,  
role[Judge|Advocate|Registrar|Admin|Public], bar_council_id?, created_at)  
  
Court (id, name, location, jurisdiction_type)  
  
Courtroom (id, court_id FK, room_number, capacity)  
  
Judge (id, user_id FK, court_id FK, designation, is_active)  
  
Case (id, case_number [unique, indexed], case_type, filing_advocate_id FK-  
>User,  
      status[Filed|Verification|Returned|Listed|Heard|Disposed],  
      priority_score (computed, indexed), is_emergency (bool),  
      filed_at, court_id FK, assigned_judge_id FK->Judge nullable)  
  
Party (id, case_id FK, name, role[Petitioner|Respondent], contact_info)  
  
Hearing (id, case_id FK, courtroom_id FK, judge_id FK, scheduled_at,  
          status[Scheduled|Completed|Adjourned|Cancelled], order_summary,  
          next_hearing_date)  
  
Document (id, case_id FK, uploaded_by FK->User,  
           doc_type[Petition|Evidence|Order|Affidavit],  
           storage_url, version, checksum, virus_scan_status, uploaded_at)  
  
PriorityFactor (id, case_id FK,  
                factor_type[Age|StatutoryUrgency|EmergencyFlag|VulnerabilityFlag],  
                weight, computed_score, computed_at)  
  
AuditLog (id, entity_type, entity_id, action, actor_id FK->User, before_state  
JSONB,  
           after_state JSONB, reason, prev_hash, hash, created_at)  -- hash-  
chained for tamper evidence  
  
Notification (id, user_id FK, case_id FK, type, channel[Email|SMS|Push],  
              sent_at, read_at)
```

Key relationships: Case 1–N Hearing, Case 1–N Document, Case 1–N Party, Case 1–N PriorityFactor, Court 1–N Courtroom, Court 1–N Judge, every mutating action → 1 AuditLog row.

2.3 Caching / Queue Strategy

- **Redis cache-aside** for: today's cause list per court (TTL 5 min, invalidated on any hearing mutation), public case-status lookups.
 - **Priority queue** implemented as a Redis **sorted set** (`ZADD court:{id}:queue {priority_score} {case_id}`) for $O(\log N)$ inserts/reorders — this is a strong talking point in interviews (heap-backed priority queue at scale, vs. recomputing SQL `ORDER BY` on every request).
 - **BullMQ jobs**: `recompute-priority` (nightly + on-event), `generate-cause-list-pdf`, `notify-fanout`, `document-virus-scan`.
 - **WebSocket rooms** per court (`court:{id}`) and per user — registrar reorders queue → emits `queue:updated` → judge dashboards patch in place without refetch.
-

3. Backend API Endpoint Design (REST)

Auth

POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout

Cases

POST /api/cases (advocate: file new case)
GET /api/cases/:id (role-scoped detail)
GET /api/cases?status=&court=&q= (search/list, paginated)
PATCH /api/cases/:id/status (registrar: verify/return)
POST /api/cases/:id/emergency-flag (advocate/registrar: mark urgent + justification)
GET /api/cases/:id/audit-trail (immutable history)

Priority Queue

GET /api/courts/:courtId/queue (current sorted queue with score breakdown)
POST /api/courts/:courtId/queue/reorder (registrar override, requires reason)
POST /api/scheduler/recompute (trigger recompute job)

Hearings / Courtroom Allocation

POST /api/hearings (registrar/judge: schedule)
PATCH /api/hearings/:id (reschedule/adjourn/dispose)
GET /api/courtrooms/:id/availability

Documents

POST /api/cases/:id/documents (multipart upload -> presigned S3 flow)
GET /api/documents/:id/download (signed URL, access-checked)

Public

GET /api/public/cause-list?court=&date=
GET /api/public/case-status?caseNumber=

Realtime (Socket.IO namespaces)

/ws/court/:courtId -> queue:updated, hearing:scheduled, emergency:alert

Auth on every non-public route via JWT + role guard; row-level checks (e.g., advocate can only see their own filed cases; judge only their assigned court).

4. Standout Portfolio Enhancements

1. **Real-time WebSocket Hearing Dockets** — live queue reordering, emergency alerts pushed instantly to judge/registrar dashboards without polling. Demonstrates event-driven architecture.
2. **AI-Assisted Petition Categorization** — use an LLM (or a lightweight classifier) to auto-suggest `case_type` and urgency class from the filed petition text, with a human-in-the-loop confirm step. Strong "applied AI" resume line, and pairs naturally with the priority-scoring engine.
3. **Explainable Weighted Priority Engine** — expose the score breakdown (age + urgency + emergency + vulnerability weights) as a visual "why is this case #3" tooltip. Signals algorithmic design skill, not just CRUD.
4. **Tamper-Evident Audit Log** — hash-chained audit trail (each entry stores hash of previous entry), verifiable integrity check endpoint. Strong systems-design/security talking point.
5. **PDF Cause-List/Order Generation** — server-side PDF generation (e.g., Puppeteer or a templating lib) for daily cause lists and signed order exports.

Pick 2-3 to build deeply rather than all 5 shallowly — depth reads better in interviews than breadth.

5. Phase-by-Phase Development Plan

Phase 1 — MVP (4-6 weeks)

- Auth + RBAC (4 roles), basic Postgres schema, Prisma/JPA setup
- Advocate: file case, upload documents (local/S3)
- Registrar: verify/return filing, manual hearing scheduling
- Judge: view docket, record disposition
- Public: case status lookup, static cause list
- Basic audit log (no hashing yet)
- Deployed on a single Docker Compose stack (Postgres + API + frontend)

Phase 2 — Core Differentiators (4-6 weeks)

- Weighted priority scoring engine + Redis sorted-set queue
- Emergency override workflow (state machine + SLA timers)
- WebSocket live docket updates
- Hash-chained tamper-evident audit log + verification endpoint
- PDF cause-list generation job (BullMQ)
- Notification fan-out (email at minimum)
- Test coverage: unit (scoring logic), integration (API + DB), one E2E flow (Playwright/Cypress)

Phase 3 — Advanced, Cloud-Ready (4-6 weeks)

- AI-assisted petition categorization (LLM API integration, human confirm step)
 - Move search to Elasticsearch/OpenSearch for public lookup at scale
 - Kubernetes manifests (or ECS) + Terraform IaC; GitHub Actions CI/CD to a real cloud environment
 - Observability: Prometheus/Grafana dashboards, Sentry error tracking, structured logging
 - Load testing (k6) on the priority-queue and public-lookup endpoints; document results
 - Security pass: rate limiting, input validation hardening, dependency scanning (Dependabot/Snyk), signed URL expiry review
 - Polish: recorded demo video, architecture diagram, README with setup + design-decision write-up (this last part matters as much as the code for portfolio impact)
-

Suggested Repo Structure for the Write-Up

Recruiters skim READMEs — lead with an architecture diagram, the priority-scoring explanation, and a live demo link/GIF before implementation details. -e

6-7. AI/LLM Integration Blueprint

Senior AI Architect Perspective

1. Recommended AI Features & Use Cases

1.1 AI Petition Categorization & Data Extraction

Goal: Ingest a filed petition PDF and auto-extract structured fields: Case Title, Petitioner/Respondent names, Case Type (Civil/Criminal/Constitutional/Family/etc.), key dates (incident date, filing date, statutory deadlines), and relief sought.

Pipeline:

1. **OCR / text extraction** — `pdfplumber`/`PyMuPDF` for digitally-native PDFs; **Tesseract OCR** (or a cloud OCR like AWS Textract / Google Document AI) as fallback for scanned/handwritten filings.
2. **Structured extraction** — pass extracted text to an LLM with **function calling / structured output** (OpenAI `response_format: json_schema`, or Gemini's structured output mode) so the model returns a strict JSON object matching your `Case`/`Party` schema fields — not free text you have to regex-parse.
3. **Validation layer** — run extracted dates/names through deterministic checks (regex for date formats, fuzzy-match party names against existing `User`/`Party` records) before writing to DB.

4. **Human-in-the-loop confirm** — registrar sees a pre-filled form with AI-suggested values highlighted; must confirm/edit before it's committed. This is important both practically (legal filings can't silently trust AI) and as an interview talking point (you understand AI needs a human checkpoint in high-stakes domains).

1.2 Intelligent Priority Score Estimator

Goal: Read the petition narrative and suggest an urgency signal to feed into the weighted priority algorithm from the core blueprint — not replace it.

Approach:

- Treat this as a **classifier + rationale generator**, not a black-box number. Prompt the LLM to output:

```
{ urgency_class: "high|medium|low", rationale: string, flagged_factors: ["irreparable harm", "minor involved", "custodial detention", ...] }
```

.
- Map `urgency_class` + `flagged_factors` to a **bounded weight contribution** (e.g., ± 15 points) inside your existing deterministic `PriorityFactor` scoring — the AI never directly sets the final score. This keeps the algorithm auditable and explainable (critical for a judicial system) while still showcasing applied AI.
- Log every AI suggestion + the human-adjusted final score as a training/eval dataset for later fine-tuning or prompt iteration.
- Consider a lightweight **local classifier** (e.g., a fine-tuned small model or even a logistic regression on TF-IDF features) as a fast first-pass filter, with the LLM only called for borderline/ambiguous cases — good for demonstrating cost-conscious architecture thinking.

1.3 Smart Case Summarization (RAG)

Goal: "Summarize File" button generates a 1-page executive summary across all uploaded evidence, orders, and hearing history for a case.

RAG pipeline:

1. **Chunking** — split each document (petition, evidence, prior orders, hearing minutes) into ~500-1000 token chunks with overlap, tagged with metadata (`case_id`, `doc_type`, `date`, `page`).
2. **Embedding** — embed chunks with an embedding model (OpenAI `text-embedding-3-large`, Gemini `text-embedding-004`, or a local model like `bge-large-en` via `sentence-transformers` if you want an all-open-source stack).
3. **Vector store** — store embeddings scoped by `case_id` (see Section 2.2 for DB choice).
4. **Retrieval** — on "Summarize File" click, retrieve top-K relevant chunks (filtered by `case_id`, optionally re-ranked with a cross-encoder for precision).
5. **Generation** — feed retrieved chunks + a structured prompt ("You are assisting a judge; produce a neutral 1-page summary organized under Parties, Timeline, Key

Evidence, Prior Orders, Open Issues") to the LLM (GPT-4-class or Gemini 1.5/2.x for long context).

6. **Citations** — have the model return which source chunk/page backs each summary claim, and render those as clickable references back to the original document — this is what separates a toy RAG demo from a production-grade one, and it's a major resume differentiator.

1.4 AI Stenographer / Transcript Summarizer

Goal: Convert raw hearing audio/text into a structured draft court order.

Pipeline:

1. **Speech-to-text** — Whisper (OpenAI API or self-hosted `whisper.cpp`/`faster-whisper` for an open-source angle) with speaker diarization (e.g., `pyannote.audio`) to separate judge/advocate/witness speech.
 2. **Structuring** — pass the diarized transcript to an LLM prompted to output a **draft order template**: case number, appearances, submissions summary, judge's directions, next hearing date — again as structured JSON/sections, not free prose, so it maps cleanly onto your `Hearing.order_summary` field.
 3. **Draft-only, never auto-published** — the draft is explicitly marked `AI-generated draft – pending judicial review` and only becomes an official order after the judge edits/approves it. State this constraint clearly in your README — it shows judgment about where AI belongs in a legal workflow, which recruiters/interviewers will specifically probe.
-

2. Technical AI Architecture

2.1 Model/Tool Selection Per Feature

Feature	Suggested Model(s)	Supporting Tools
OCR / doc text extraction	Tesseract OCR (open-source) or Google Document AI / AWS Textract (better accuracy on scans)	<code>pdfplumber</code> , <code>pdf2image</code>
Structured extraction (categorization)	GPT-4o / GPT-4.1 with structured outputs, or Gemini 1.5/2.x with JSON mode	LangChain <code>with_structured_output</code> , or raw API + Pydantic/Zod schema validation
Priority urgency classification	Small/cheap model first pass (GPT-4o-mini, Gemini Flash) → escalate to a larger model for ambiguous cases	Simple rules engine as a pre-filter
RAG summarization	Long-context model: GPT-4.1, Gemini 1.5/2.x Pro (huge context windows are genuinely useful here), or Llama 3.1/3.3 self-hosted via vLLM/Ollama for a fully open-source stack story	LangChain or LlamaIndex for retrieval orchestration
Embeddings	<code>text-embedding-3-large</code> (OpenAI), <code>text-embedding-004</code> (Gemini), or <code>bge-large-en-v1.5</code> / <code>nomic-embed-text</code> (open-source, run locally)	<code>sentence-transformers</code>
Speech-to-text	Whisper API or self-hosted <code>faster-whisper</code>	<code>pyannote.audio</code> for diarization

Framework choice: LangChain or LlamaIndex both work; LlamaIndex tends to be more RAG-focused and slightly less boilerplate for document indexing, while LangChain has a broader ecosystem (agents, tool-calling chains) if you plan to expand beyond RAG later (e.g., an agent that cross-references case law). Either is a reasonable resume line — pick one and go deep rather than mixing both.

Local LLM option (Llama 3): If you want to demonstrate you can run AI without depending on paid APIs (a strong signal for cost-conscious/infra-aware roles), self-host Llama 3.1/3.3 8B or 70B via **Ollama** (simplest) or **vLLM** (production-grade, better throughput) behind an internal API that mimics the OpenAI SDK shape — this lets you swap providers with minimal code change and is worth explicitly calling out in your README as a "provider-agnostic AI layer" design decision.

2.2 Vector Database Setup for RAG

Option	When to use
PGVector (PostgreSQL extension)	Recommended for this project. You already run Postgres for the core app — adding <code>pgvector</code> avoids a second database to operate, keeps embeddings joined naturally to <code>case_id</code> / <code>document_id</code> via foreign keys, and is a great "I made a pragmatic architecture decision" talking point instead of over-engineering with a separate vector DB.
ChromaDB	Good for local dev/prototyping or if you want a lightweight, embeddable vector store with zero infra ops.
Pinecone	Use if you want to demonstrate a managed, horizontally-scalable vector search service — a stronger "cloud-native at scale" story for Phase 3, but adds an external dependency/cost.

Recommended path: Start with **PGVector** for Phase 1/2 (simplicity, single source of truth, transactional consistency between case data and embeddings), and mention Pinecone/OpenSearch-with-kNN as the documented "scale-out path" in your README for when corpus size grows — this shows you can reason about scaling without over-building prematurely.

Schema addition:

```
sql

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunk (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID REFERENCES "Document"(id),
  case_id UUID REFERENCES "Case"(id),
  chunk_text TEXT NOT NULL,
  page_number INT,
  embedding VECTOR(1536), -- match your embedding model's dimension
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON document_chunk USING ivfflat (embedding vector_cosine_ops) WITH
```

Retrieval query pattern: filter by `case_id` first (cheap index scan), then order remaining rows by cosine distance to the query embedding — this two-stage filter-then-rank is worth

explaining in an interview since naive vector search often skips the metadata pre-filter and ends up needlessly expensive.

2.3 Architectural Guardrails Worth Documenting

- **Cost/latency tiering:** cheap model for classification/extraction, expensive model only for generation-heavy tasks (summarization, draft orders).
 - **Prompt versioning:** store prompts in version-controlled files (not inline strings scattered in code), log which prompt version produced which output — useful for debugging and shows engineering maturity.
 - **Evaluation harness:** even a small set of 15–20 hand-labeled petitions to test extraction accuracy against is a strong artifact to show in a portfolio ("here's how I measured my AI feature's precision/recall," not just "it works on my demo").
 - **Human review is non-negotiable everywhere AI touches judicial output** — extraction, priority suggestions, and draft orders are all reviewable/editable before they affect real workflow state. This is both the ethically correct design for this domain and the single most interview-worthy design decision in the whole AI layer.
-

Suggested Build Order

1. OCR + structured extraction (highest immediate value, lowest complexity)
2. PGVector + basic RAG summarization
3. Priority urgency classifier (bounded contribution to existing algorithm)
4. Speech-to-text + draft order generation (most complex, save for last)